

TESTES

Get (decrecente)

The screenshot shows a REST client interface with a GET request to `http://localhost:3000/produtos`. The response is a JSON object with a status of 200 OK, size of 243 Bytes, and time of 20 ms. The response body is a JSON array with two objects, each representing a product.

```
1 {
2   "sucesso": true,
3   "dados": [
4     {
5       "id": 2,
6       "nome": "shampoo capilar",
7       "descricao": "produto que lava cabelos",
8       "preco": "9.99",
9       "disponivel": 45
10    },
11    {
12      "id": 1,
13      "nome": "minoxidil",
14      "descricao": "produto que faz crescer pelos",
15      "preco": "40.00",
16      "disponivel": 8
17    }
18  ],
19   "total": 2
20 }
```

Get (id)

The screenshot shows a REST client interface with a GET request to `http://localhost:3000/produtos/1`. The response is a JSON object with a status of 200 OK, size of 127 Bytes, and time of 22 ms. The response body is a JSON object representing a single product.

```
1 {
2   "sucesso": true,
3   "dados": {
4     "id": 1,
5     "nome": "minoxidil",
6     "descricao": "produto que faz crescer pelos",
7     "preco": "40.00",
8     "disponivel": 8
9   }
10 }
```

POST

POSThttp://localhost:3000/produtosSend

QueryHeaders 2AuthBody 1TestsPre Run

JSONXMLTextFormForm-encodeGraphQLBinary

JSON ContentFormat

```
1 {
2   "id": 2,
3   "nome": "shampoo capilar",
4   "descricao": "produto que lava cabelos",
5   "preco": "9.99",
6   "disponivel": 45
7 }
8
```

Status: 201 CreatedSize: 63 BytesTime: 28 ms

ResponseHeaders 6CookiesResultsDocs

```
1 {
2   "sucesso": true,
3   "mensagem": "Produto criado com sucesso",
4   "id": 2
5 }
```

ResponseChart

PUT

PUThttp://localhost:3000/produtos/2Send

QueryHeaders 2AuthBody 1TestsPre Run

JSONXMLTextFormForm-encodeGraphQLBinary

JSON ContentFormat

```
1 {
2   "nome": "shampoo capilar",
3   "descricao": "produto que lava cabelos",
4   "preco": 50.00,
5   "disponivel": 100
6 }
7
```

Status: 200 OKSize: 60 BytesTime: 26 ms

ResponseHeaders 6CookiesResultsDocs

```
1 {
2   "sucesso": true,
3   "mensagem": "Produto atualizado com sucesso"
4 }
```

ResponseChart

DELETE

The screenshot displays a REST client interface with a DELETE request to `http://localhost:3000/produtos/2`. The request body is a JSON object: `{ "nome": "shampoo capilar", "descricao": "produto que lava cabelos", "preco": 50.00, "disponivel": 100 }`. The response is a 200 OK status with a 58-byte body: `{ "sucesso": true, "mensagem": "Produto removido com sucesso" }`.

Method	URL	Status	Size	Time
DELETE	http://localhost:3000/produtos/2	200 OK	58 Bytes	23 ms

Request Body (JSON):

```
1 {
2   "nome": "shampoo capilar",
3   "descricao": "produto que lava cabelos",
4   "preco": 50.00,
5   "disponivel": 100
6 }
```

Response Body (JSON):

```
1 {
2   "sucesso": true,
3   "mensagem": "Produto removido com sucesso"
4 }
```

O Node.js com Express é adequado para uma API REST porque é rápido e assíncrono, permitindo lidar com várias requisições ao mesmo tempo sem travar. Além disso, o Express facilita a criação de rotas e o desenvolvimento do backend, usando JavaScript de forma simples e eficiente, além de integrar bem com bancos de dados como o MySQL.